

Documentação Técnica: Próxima Sessão

O **Próxima Sessão** é uma aplicação web voltada para a descoberta e organização de filmes e séries. O projeto foi desenvolvido com foco na simplicidade da arquitetura, dispensando o uso de bibliotecas complexas para gerenciamento de estado global ou requisições HTTP, baseando-se em recursos nativos do navegador sempre que possível.

Tecnologias e Ferramentas

- **Core:** React, Vite, JavaScript, CSS tradicional (puro).
- **Roteamento:** `react-router-dom`.
- **Ícones:** `lucide-react`.
- **Integração HTTP:** `fetch` nativo com `AbortController`.
- **Armazenamento:** `localStorage` nativo.
- **O que NÃO foi utilizado:** Context API, Redux, Axios, bancos de dados, autenticação ou frameworks CSS (como Tailwind ou Bootstrap).

Nota: Dependências de produção limitam-se ao React, React DOM, React Router e Lucide. Vite e ESLint operam apenas em ambiente de desenvolvimento.

Arquitetura e Estrutura de Pastas

A base do código está em `src/`, dividida de forma a evitar abstrações desnecessárias:

- `src/components/`: Componentes visuais isolados (Header, Footer, Poster, MovieCard, MovieGrid, SearchBar, Feedback, PopularSection).
- `src/pages/`: Componentes de nível de rota (Home, Search, Details, MyList).
- `src/layouts/`: Estruturas de página, especificamente o `MainLayout.jsx`.
- `src/services/`: Lógica externa aos componentes (`api.js` para comunicação com a TMDB e `storage.js` para lidar com a lista local).

O repositório ignora arquivos sensíveis (como `.env.local`), módulos instalados e a pasta de build. A pasta `template_ref` contém material de referência e não é processada pela aplicação.

Roteamento e Navegação

As rotas são definidas via `BrowserRouter` (em `main.jsx`) e gerenciadas por `Routes` e `Route` (em `App.jsx`).

- `/` : **Home** - Exibe as listas de filmes e séries populares.
- `/buscar` : **Busca** - Interface de pesquisa por título.

- `/detalhes/:type/:id` : **Detalhes** - Informações completas de um título. O parâmetro `type` aceita `movie` ou `tv`, e o `id` deve ser numérico (ex: `/detalhes/movie/550`).
- `/minha-lista` : **Minha Lista** - Títulos salvos pelo usuário.
- **Rota 404 (Fallback)**: Qualquer URL não mapeada mantém o layout principal e exibe o componente `Feedback` com um link para voltar ao início.

Comportamentos de Navegação:

- O `MainLayout` usa `useLocation` para interceptar mudanças de rota, forçando o scroll para o topo e movendo o foco para o conteúdo principal visando acessibilidade.
- Na página de detalhes, o componente interno `DetailsContent` recebe uma `key` atrelada à URL. Isso força a remontagem completa do componente ao navegar entre dois títulos diferentes, impedindo que dados do título anterior "pisquem" na tela durante o carregamento.

Gerenciamento de Estado e Storage

O estado global da aplicação (a lista de títulos salvos) fica centralizado no `App.jsx`. Isso permite que qualquer página saiba se um título está salvo sem precisar de Context API.

Fluxo do LocalStorage (`proxima-sessao-lista`):

1. A leitura inicial ocorre usando `JSON.parse`. Se os dados estiverem corrompidos, a aplicação trata o erro para não quebrar a tela, e a leitura é bloqueada (`readBlocked`).
2. Nunca ocorre substituição silenciosa de dados que falharam na leitura inicial.
3. Quando o usuário adiciona ou remove um item, a alteração vai para uma fila pendente (`pendingList`).
4. A interface só reflete a mudança de forma definitiva após o `storage.js` confirmar a gravação no disco físico.
5. Se a gravação falhar (ex: armazenamento cheio), a lista anterior é mantida e uma mensagem de erro é exibida via `storageError`.

A página **MyList** consome esse estado e a função de remoção. A página **Details** consome a lista para calcular dinamicamente se o título atual já está salvo usando `list.some(...)`, evitando duplicidade de estados.

Integração com a API (TMDB)

As requisições ficam isoladas em `api.js` e apontam para `[https://api.themoviedb.org/3](https://api.themoviedb.org/3)`. Chave de acesso: Lida via `VITE_TMDB_API_KEY` (configurada no `.env`). Como se trata de SPA, essa chave é pública no navegador.

Endpoints Utilizados:

- `/movie/popular` e `/tv/popular` (Home)
- `/search/multi` (Busca filtrada apenas para filmes e séries, sem conteúdo adulto)
- `/movie/:id` e `/tv/:id` (Detalhes)

Tratamento e Normalização: A API padroniza as respostas recebidas em um objeto único. Campos vazios ou faltantes não são inventados; recebem tratamentos padrão:

- `id` e `type` (Identificadores)
- `title` (Filmes usam `title`, séries usam `name`. Fallback: "Título indisponível").
- `poster` (Nulo se ausente. Puxado de `[https://image.tmdb.org/t/p/w500](https://image.tmdb.org/t/p/w500)`).
- `year` (Extraído de `release_date` ou `first_air_date`. String vazia se ausente).
- `rating` e `voteCount` (A nota só é exibida se `voteCount > 0`).
- `overview` e `genres` (String ou array vazio se ausentes).

Prevenção de Race Conditions: Componentes como `PopularSection`, `SearchResults` e `DetailsContent` possuem efeitos (`useEffect`) para buscar os dados. Caso o usuário saia da tela antes da requisição terminar, o resultado é ignorado, evitando que promessas antigas sobrescrevam estados mais recentes. O tempo limite (`setTimeout` + `AbortController`) é cravado em 15 segundos.

Componentes Principais e Props

A passagem de dados é estritamente via *props*. Funções de evento recebem o valor final (ex: `onAdd(movie)` recebe o objeto inteiro, `onChange(texto)` recebe a string do input).

- **Header / Footer:** Interface estática. Footer inclui obrigatoriamente os créditos da TMDB.
- **Poster:** Recebe `path` e `title`. Possui estado interno (`failedPath`) que exibe um layout alternativo caso o link da imagem quebre.
- **MovieCard:** Recebe `movie` e opcionalmente `onRemove`. Monta o link para a página de detalhes unindo `type` e `id`.
- **MovieGrid:** Faz o `map()` da lista e renderiza os `MovieCards`.
- **SearchBar:** Componente controlado recebendo `value`, `onChange` e `onSubmit`. A busca só é acionada no envio (`submit`), não a cada digitação.
- **PopularSection:** Gerencia seus próprios estados de `movies`, `loading` e `error`, garantindo que o carregamento de Filmes e Séries na Home ocorram de forma independente e paralela.
- **Feedback:** Renderizador genérico de estados visuais (carregando, erro, não encontrado, lista vazia). Recebe `title`, `kind`, `message` e `children`.

Interface e Responsividade

O projeto visual dispensa bibliotecas CSS, utilizando uma folha de estilos baseada em variáveis e Media Queries.

- **Identidade Visual:** Fundo escuro (grafite), fonte nativa do sistema (sans-serif) para melhor performance, texto claro para contraste e elementos de ação/destaque em verde-lima.
- **Layout:** CSS Grid gerencia o posicionamento e alinhamento dos cards de filmes. Flexbox é utilizado na barra de navegação e em agrupamentos menores de botões/ícones.
- **Breakpoints (Media Queries):**
 - 1100px e 760px: Redução do número de colunas nos grids.
 - 480px: Ajuste severo para mobile (formulários, espaçamentos internos e reorganização da tela de detalhes).
 - Suporte mínimo planejado: Telas a partir de 320px.

Matriz de Requisitos (RF)

- **RF01 (Listagem de Populares):** Atendido pelos componentes `Home` e `PopularSection`.
- **RF02 (Busca de Títulos):** Atendido por `Search` e `SearchBar`.
- **RF03 (Exibição de Detalhes):** Atendido por `Details`, `Poster` e tratamento de erros 404 da API.
- **RF04 (Adicionar à Lista):** Fluxo capturado em `Details`, gerenciado em `App` e persistido via `storage.js`.
- **RF05 (Remover da Lista):** Fluxo disparado a partir da rota de `MyList`, com espelho imediato no `localStorage`.
- **RF06 (Navegação SPA):** Atendido por `MainLayout` usando os recursos do `react-router-dom` sem recarregamento da página. Publicações futuras devem usar o `vercel.json` configurado para redirecionamento direto de rotas.